MINI PROJECT

Kth Largest Element in a Binary Search Tree (BST)

1. Problem Statement

Develop a C++ program to find the Kth largest element in a Binary Search Tree (BST). The program accepts distinct integer values, inserts them into a BST, and asks the user to enter one or more K values. It validates every K value and reports when K is outside the valid range. During the search, the program displays the nodes visited in reverse inorder order (Right → Root → Left), stops when the Kth largest element is found, and records how many nodes were visited. The program also compares the number of visited nodes for different K values.

2. Introduction

A Binary Search Tree is a non-linear data structure that stores values in a hierarchical form. For each node, values smaller than the node are placed in its left subtree and values greater than the node are placed in its right subtree. This arrangement supports searching and ordered processing of data.

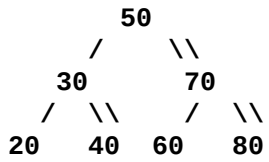
The Kth largest element is the value at position K when the values are arranged from largest to smallest. For example, K = 1 gives the largest element, while K = 2 gives the second largest. A BST can find this value using reverse inorder traversal: first visit the right subtree, then the current node, and finally the left subtree. This visits values in descending order.

In this modified project, the search displays each visited node, validates K, counts the nodes visited before finding the answer, and lets the user test several K values on the same BST. The visit counts can then be compared to observe the number of nodes needed for different K values.

3. Objectives and Suggested Modifications

- Construct a BST from distinct integer values.
- Validate K and report when it is outside the valid range (1 to n).
- Display nodes visited in reverse inorder order during the search.
- Compare the number of visited nodes for different K values.
- Practise recursion, tree traversal, and basic performance analysis.

4. BST Diagram and Example



For input values 50, 30, 70, 20, 40, 60, 80, reverse inorder traversal produces 80, 70, 60, 50, 40, 30, 20. If K = 3, the program visits 80, 70, and 60; therefore, the third largest element is 60 and 3 nodes are visited.